01: Memory-Mapped IO

Buses

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

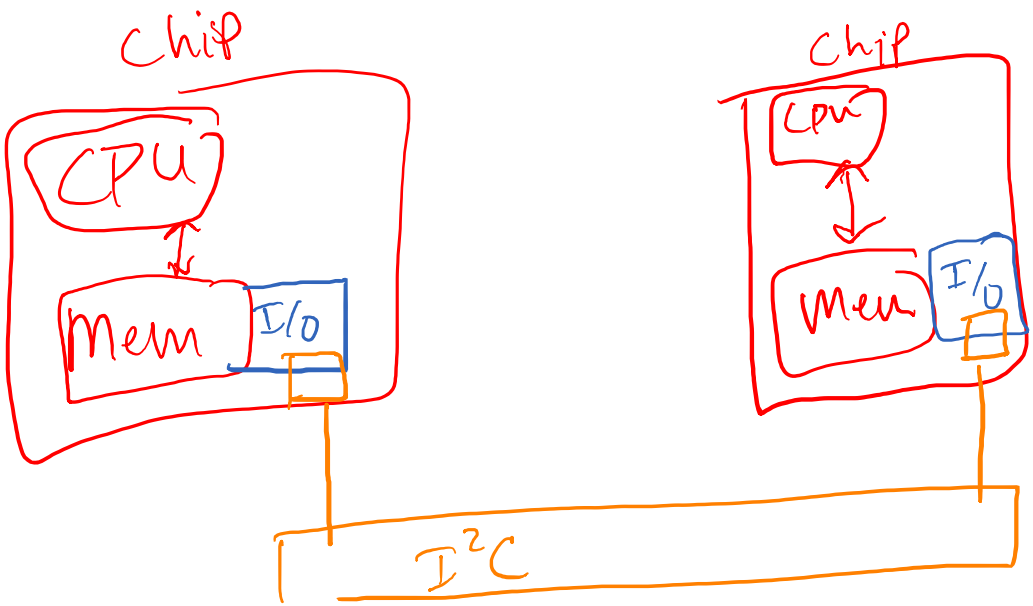
Announcements

- engr315.github.io

- Lab this Friday
- No class on Monday

Review: Memory-Mapped I/O

lets CPU communicate

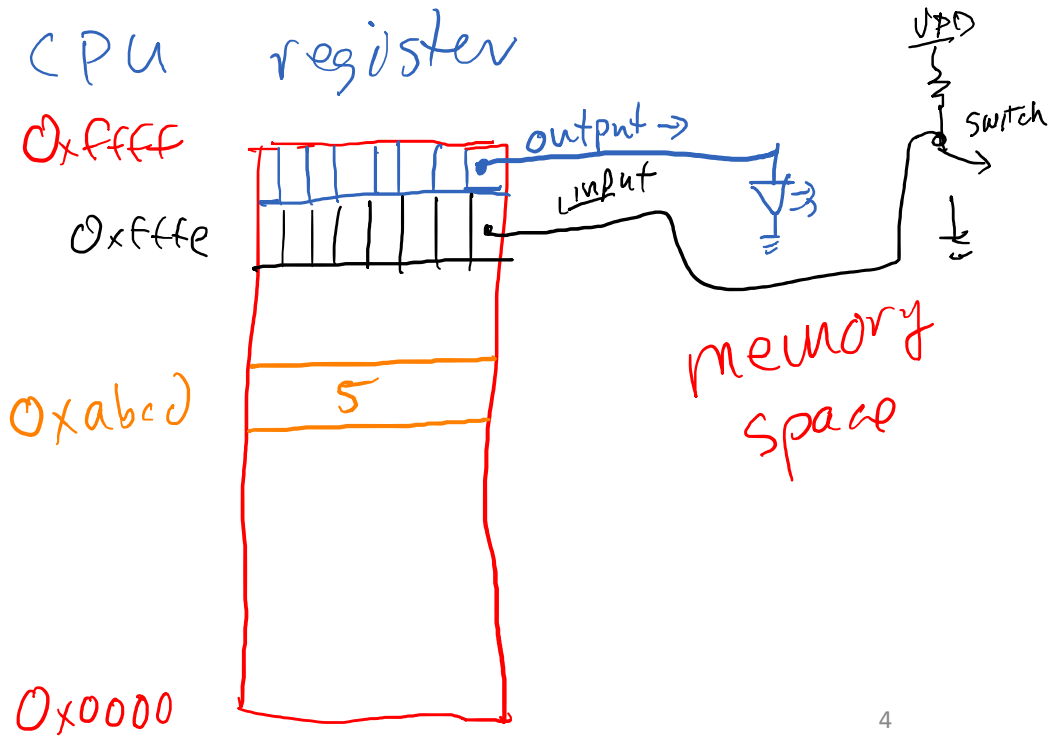


Memory-Mapped I/O

connect I/O to memory address
not CPU register

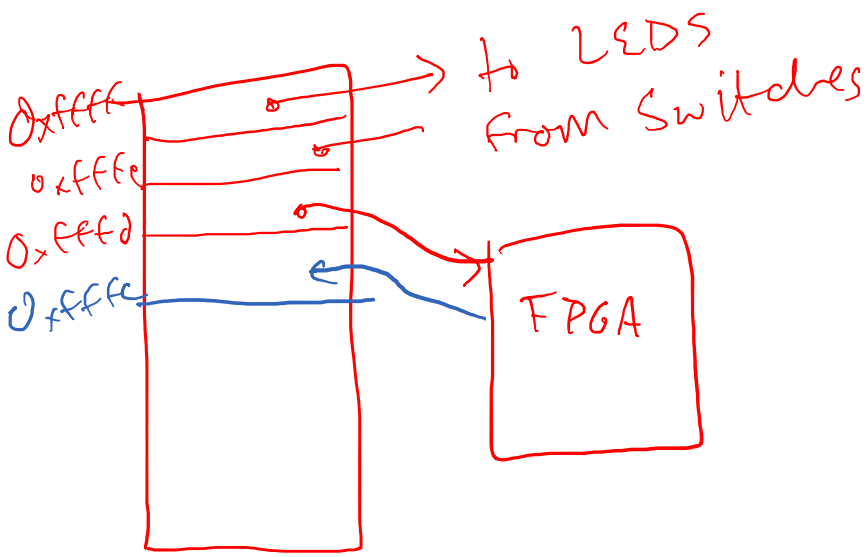
load 0xabcd → 5
store 5 ⇒ 0xabcd

store 1 ⇒ 0xffff
load 0xfffe → r0



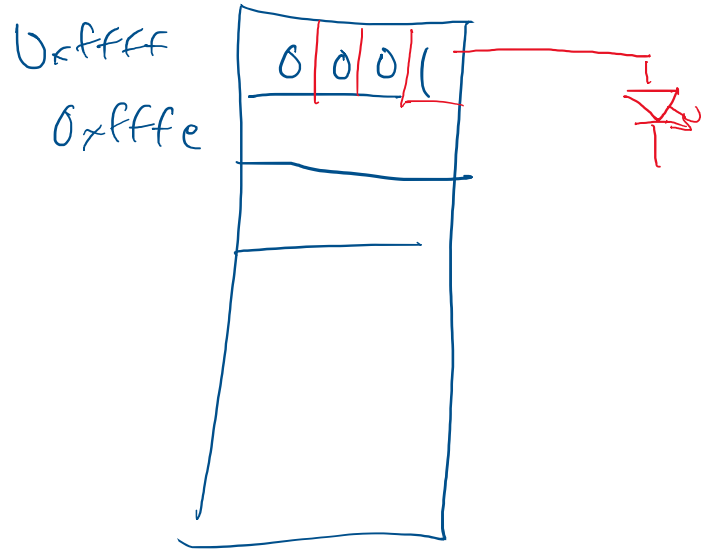
Memory-Mapped I/O

- I/O devices pretend to be memory
- Devices accessed with native CPU load/store instructions



MMIO Store from ARM Assembly

```
mov  r0, 0x1
mov  r1, 0xffff
str  r0, [r1]
```



MMIO Store from C

```
#define LED_ADDR 0xffff
uint32_t * LED_REG = (uint32_t *)(LED_ADDR);
*LED_REG = 0x1;
```

MMIO Load from ARM Assembly

```
mov r1, 0xfffe
ldr r2, [r1]
```

MMIO Load from C

volatile

```
#define SW_ADDR 0xfffe
uint32_t * SW_REG = (uint32_t *)(SW_ADDR);
int y = (*SW_REG);
```

Use `volatile` for MMIO addresses!

```
#define SW_ADDR 0xfffe
volatile uint32_t * SW_REG = (uint32_t * SW_ADDR);

int quit = (*SW_REG);
while(!quit)
{
     //more code
     quit = (*SW_REG);
}
```

`volatile` Variables

- `volatile` keyword tells compiler that the memory value is subject to change randomly.

- Use `volatile` for all MMIO memory. The values change randomly!

What happens here?

```
#include <stdio.h>
#include <inttypes.h>

#define REG_FOO 0x40000140

int main () {
  volatile uint32_t *reg = (uint32_t *)(REG_FOO);
  *reg += 3;

  printf("0x%x\n", *reg); // Prints out new value
}
```

Let’s find out…

```
vi test.c
arm-none-eabi-gcc -o test.o test.c -g
                                -O1 --specs=nosys.specs
arm-none-eabi-objdump -DS test.o > test.dis
```

Let’s find out…

```
volatile uint32_t *reg = (uint32_t *)(REG_FOO);
*reg += 3;
   8224:   e3a03101    mov r3, #1073741824 ; 0x40000000
   8228:   e5932140    ldr r2, [r3, #320]  ; 0x140
   822c:   e2822003    add r2, r2, #3
   8230:   e5832140    str r2, [r3, #320]  ; 0x140
```

What do the CPU and Memory need to communicate?

CPU → Address → Mem

← Value ←

} load

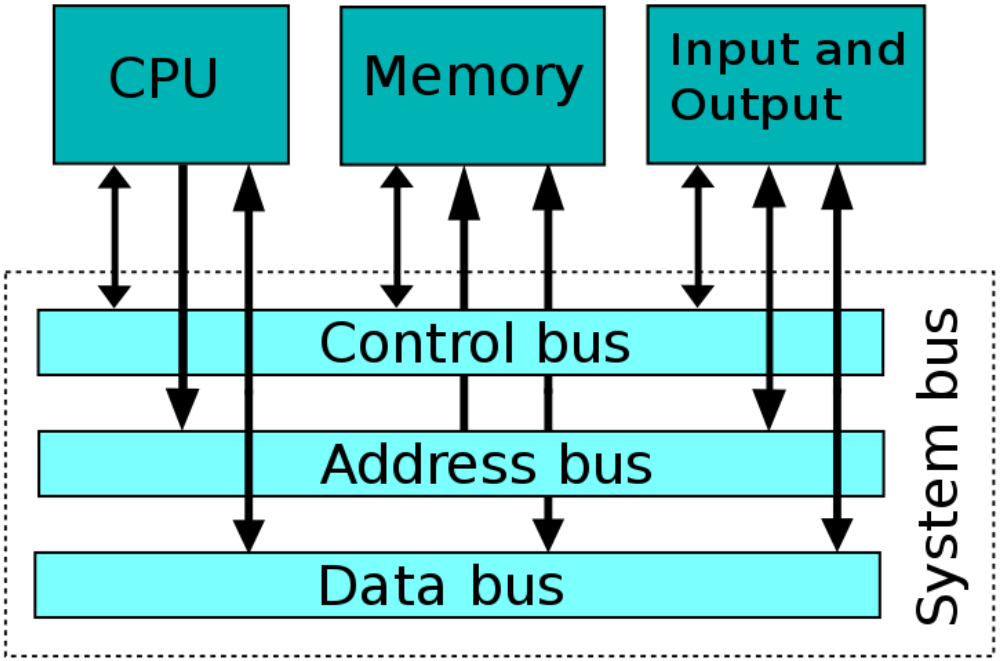
CPU → Address → Mem

Value

} Store

Store vs. Load

The System Bus



Bus terminology

Transaction: Load or Store sequence

- A "transaction" occurs between an "*initiator*" and "*target*"
- Any device capable of being an initiator is said to be a "*bus master*"
  - In many cases there is only one bus master (*single master* vs. *multi-master*).
- A device that can only be a target is said to be a "slave device".

Terminology

- Who is the bus master?

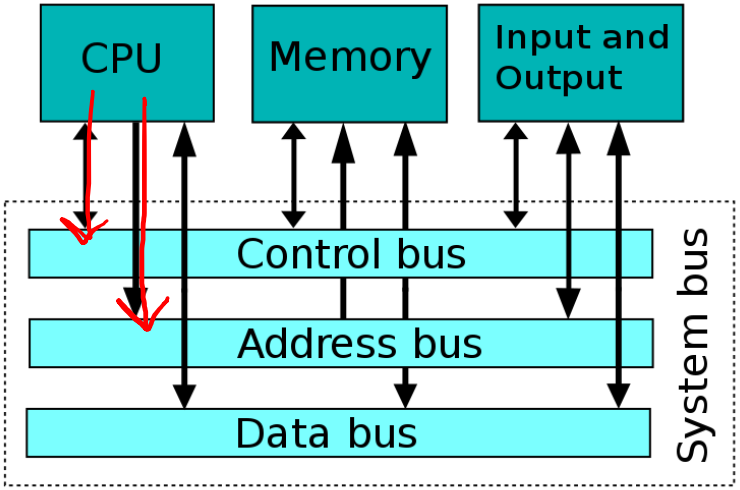
  CPU

- CPU wants to load data from Memory

  Memory puts value

- CPU wants to store data to I/O

  CPU puts value



Hypothetical Bus Example

cpu → ldr ⇒ Bus → read

cpu → str ⇒ Bus → write

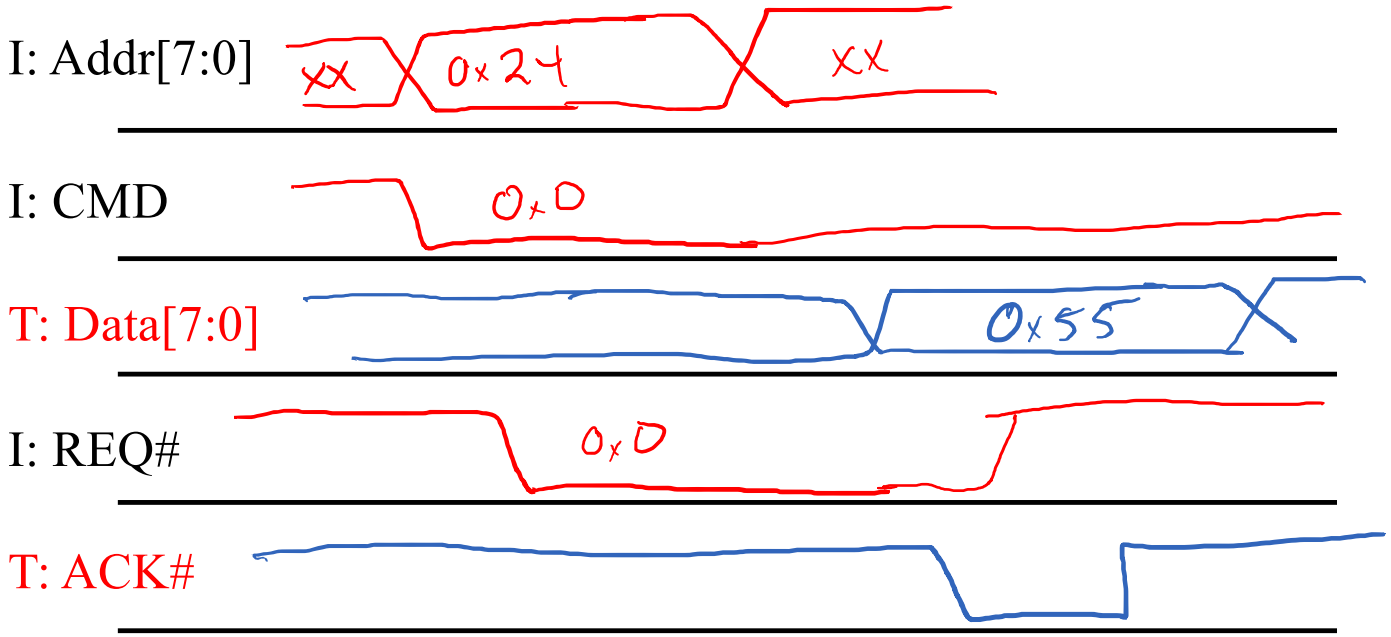
- Characteristics
  - Asynchronous (no clock) – hay, why no?
  - One Initiator and One Target

- Signals
  - Addr[7:0], Data[7:0], CMD, REQ#, ACK#
    - CMD=0 is read, CMD=1 is write.
    - REQ# low means initiator is requesting something.
    - ACK# low means target is acknowledging the job is done.

Read transaction

Initiator wants to read location 0x24

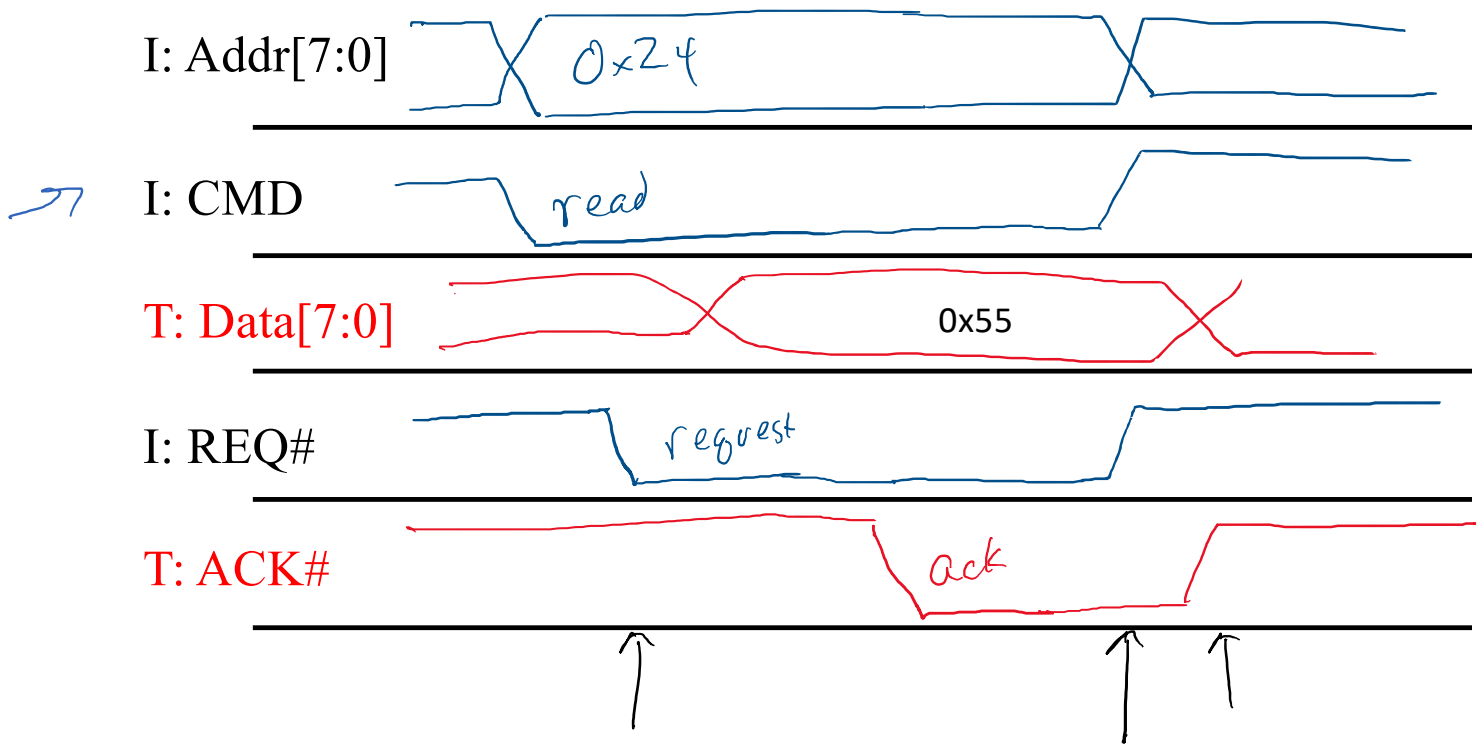
CMD=0 is read, CMD=1 is write.
REQ# low means initiator is requesting.
ACK# low means target is acknowledging.



Read transaction

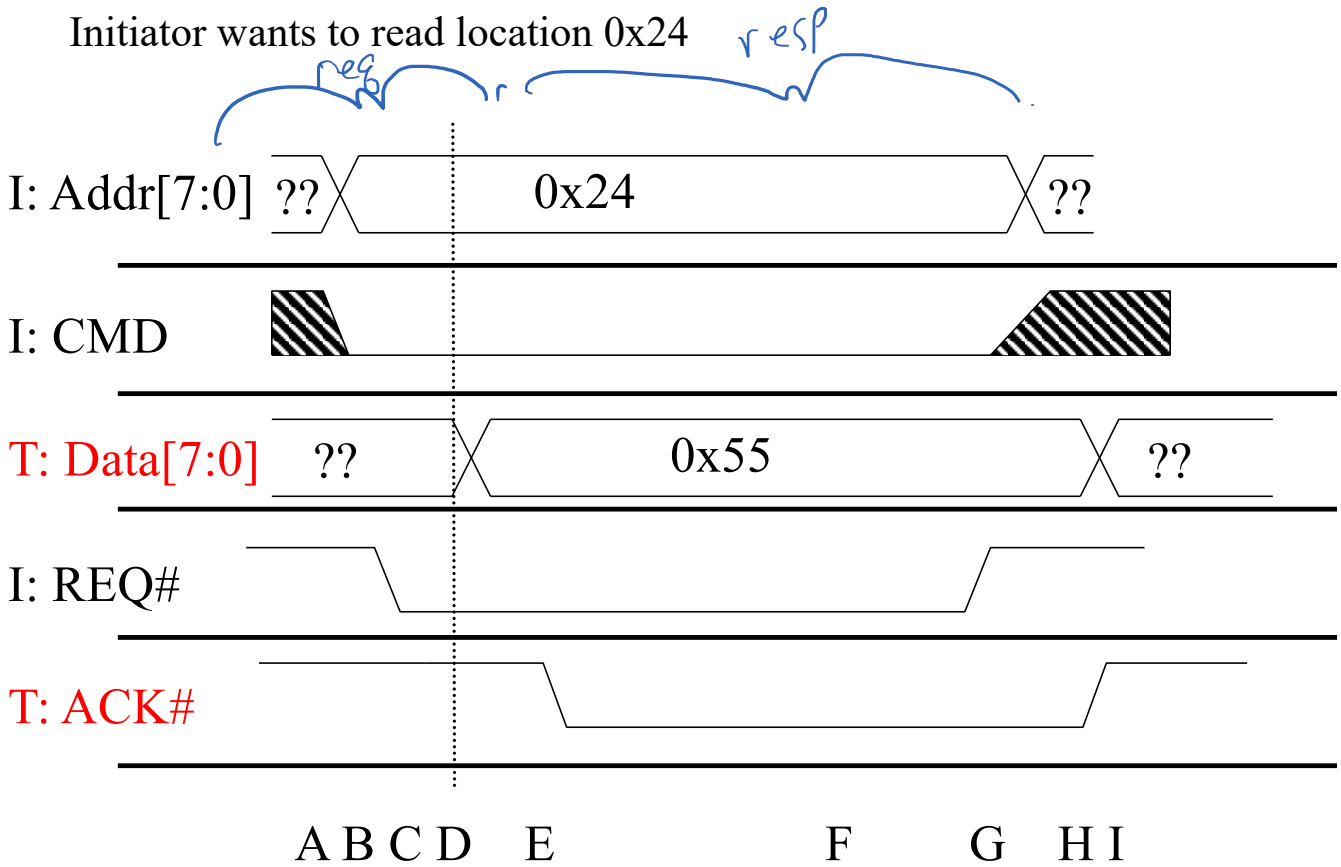
Initiator wants to read location 0x24

CMD=0 is read, CMD=1 is write.
REQ# low means initiator is requesting.
ACK# low means target is acknowledging.



Read transaction

Initiator wants to read location 0x24



A read transaction

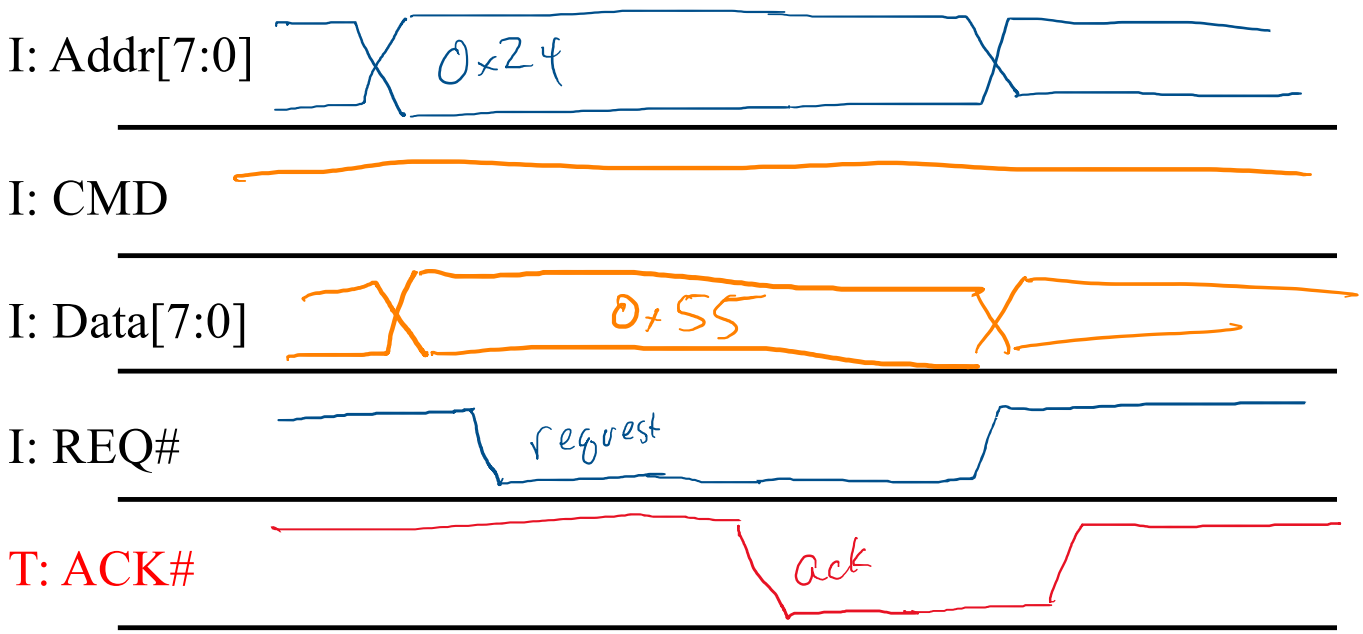
Say initiator wants to read location 0x24

A. Initiator sets Addr=0x24, CMD=0
B. Initiator *then* sets REQ# to low
C. Target sees read request
D. Target drives data onto data bus
E. Target *then* sets ACK# to low
F. Initiator grabs the data from the data bus
G. Initiator sets REQ# to high, stops driving Addr and CMD
H. Target stops driving data, sets ACK# to high terminating the transaction
I. Bus is seen to be idle

Write transaction

Initiator wants to write 0x56 to location 0x24

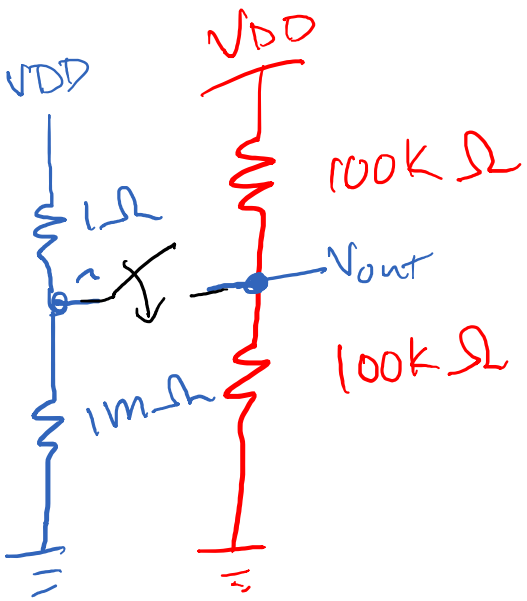
CMD=0 is read, CMD=1 is write.
REQ# low means initiator is requesting.
ACK# low means target is acknowledging.



A write transaction

- Say initiator wants to write 0xF4 location 0x31
  A. Initiator sets Addr=0x24, CMD=1, Data=0xF4
  B. Initiator *then* sets REQ# to low
  C. Target sees write request
  D. Target reads data from data bus
     (only needs to store in register, not write all the way to memory)
  E. Target *then* sets ACK# to low.
  F. Initiator sets REQ# to high, stops driving other lines
  G. Target sets ACK# to high, terminating the transaction
  H. Bus is seen to be idle.

Is MMIO really memory?



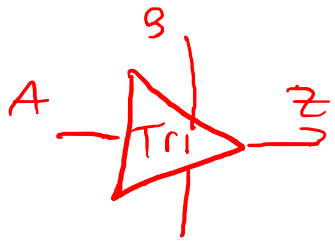
Now that we have an example bus, how would we build memory-mapped I/O?

Example peripherals

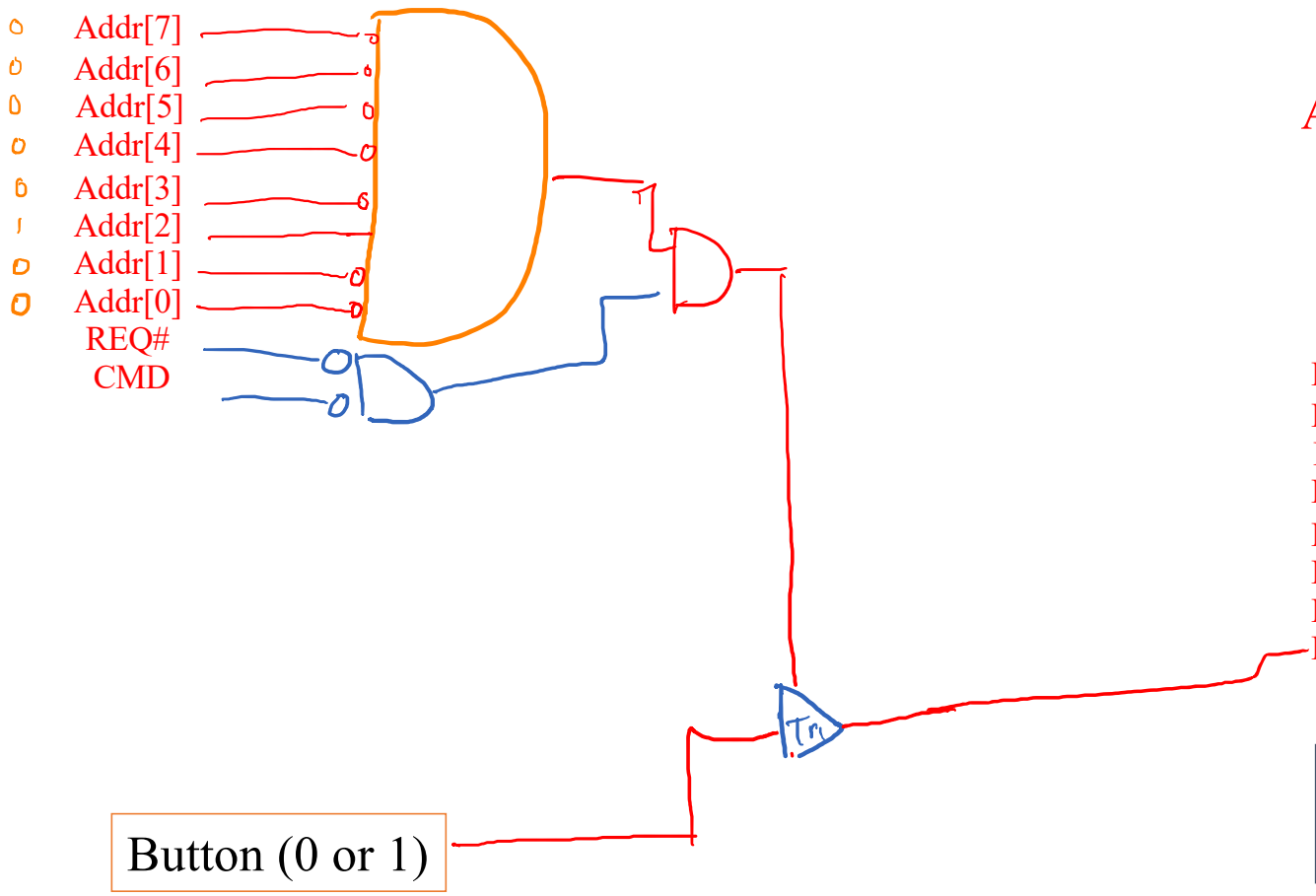
- 0x04: Push Button - Read-Only
  - Pushed -> 1
  - Not Pushed -> 0
- 0x05: LED Driver - Write-Only
  - On -> 1
  - Off -> 0

The push-button

(if Addr=0x04 write 0 or 1 depending on button)



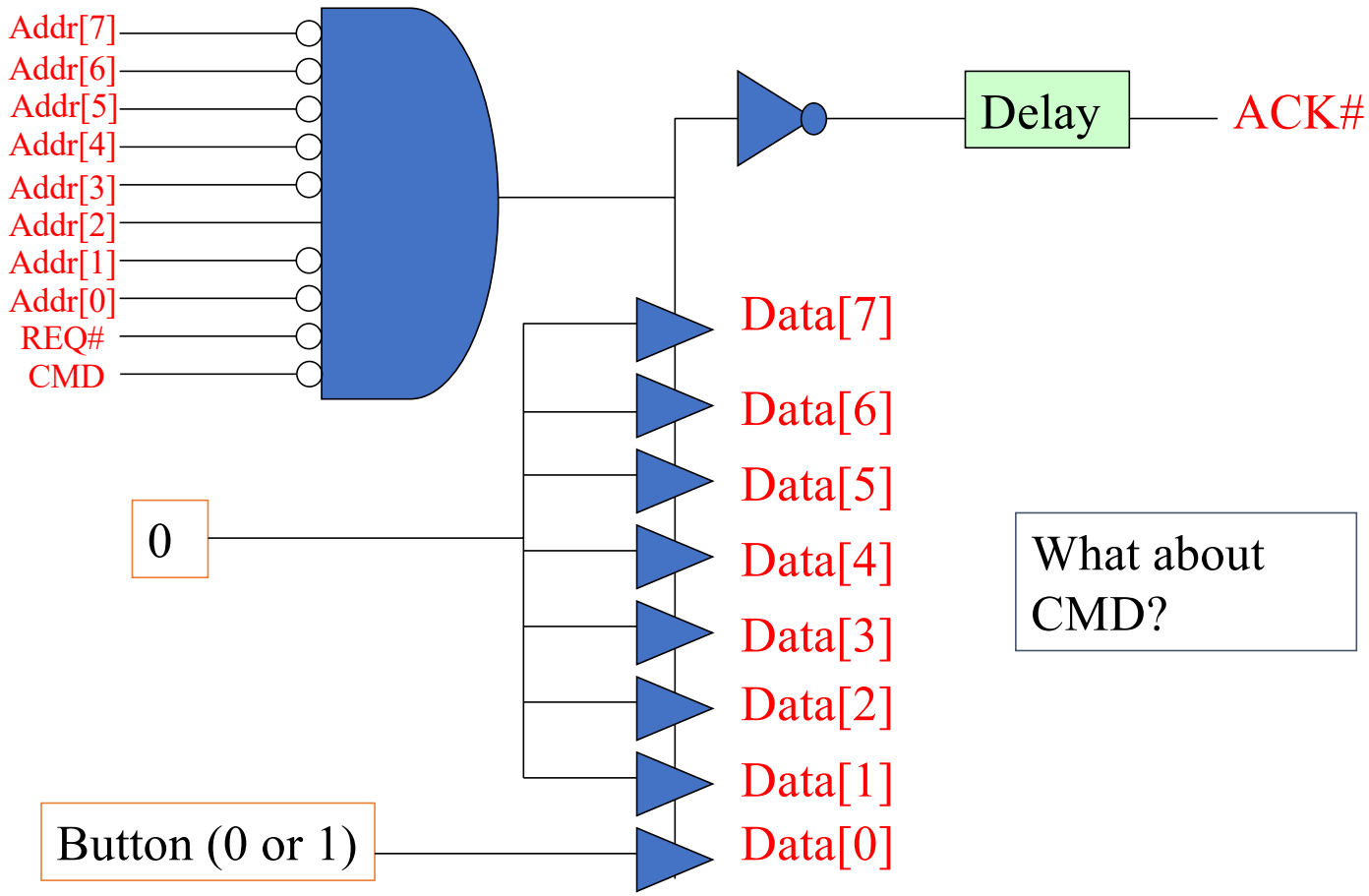
A	B	Z
0	0	High Z
0	1	0
1	0	High Z
1	1	1



What about CMD?

The push-button

(if Addr=0x04 write 0 or 1 depending on button)



The LED

(1 bit reg written by LSB of address 0x05)

Addr[7]
Addr[6]
Addr[5]
Addr[4]
Addr[3]
Addr[2]
Addr[1]
Addr[0]
REQ#
CMD

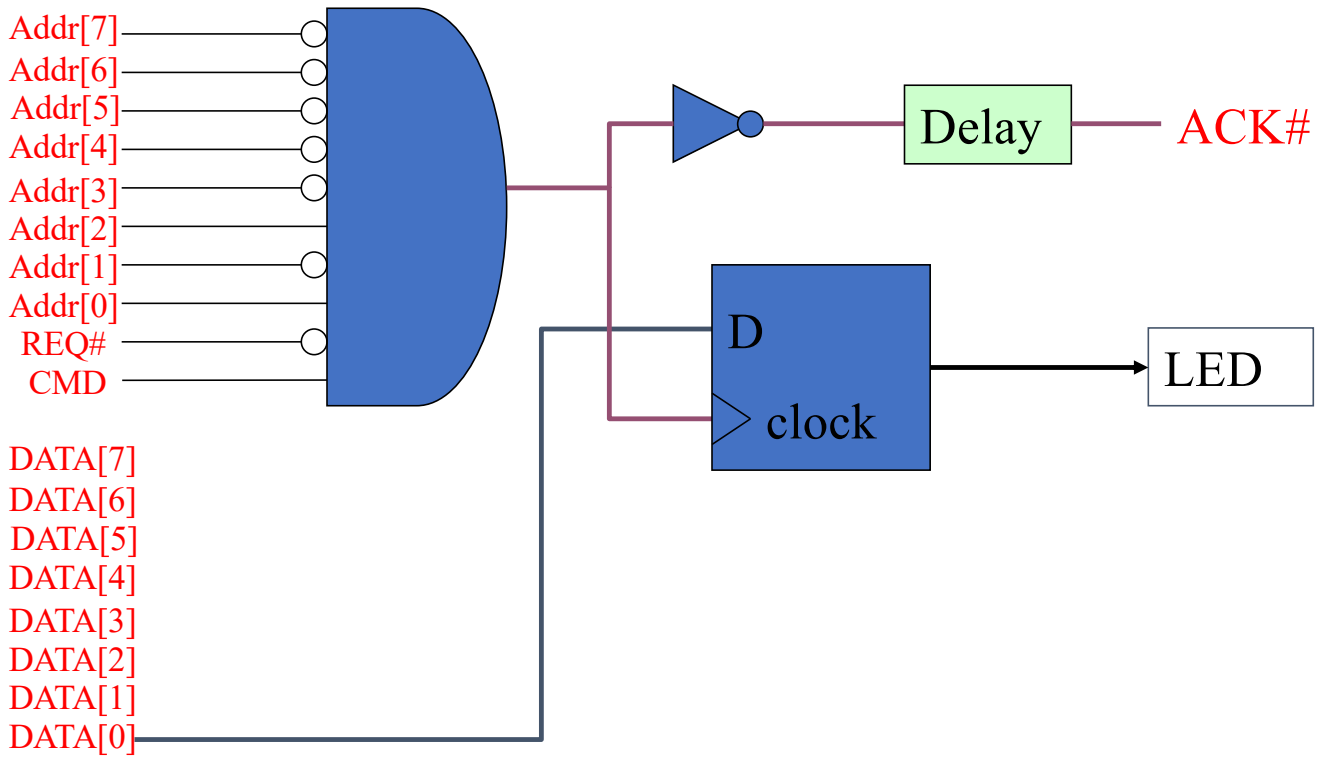
DATA[7]
DATA[6]
DATA[5]
DATA[4]
DATA[3]
DATA[2]
DATA[1]
DATA[0]

ACK#

LED

The LED

(1 bit reg written by LSB of address 0x05)



Let's write a simple assembly program
Light on if button is pressed.

Peripheral Details

- 0x04: Push Button - Read-Only
  - Pushed -> 1
  - Not Pushed -> 0
- 0x05: LED Driver - Write-Only
  - On -> 1
  - Off -> 0

```
#define PB 0x04
#define LED 0x05

int main() {
register int val;
for (;;) {
val = *(volatile uint32_t *)(PB);
*(volatile uint32_t*)(LED) = val;
}
}
```

```
        mov r0, #0x4  % PB
        mov r1, #0x5  % LED
loop:   ldr r2, [r0, #0]
        str r2 [r1, #0]
        b loop
```

Next Time

- Real On-Chip Busses
- AXI4

References

- https://www.youtube.com/watch?v=okiTzvihHRA
- https://web.eecs.umich.edu/~prabal/teaching/eecs373/
- https://en.wikipedia.org/wiki/File:Computer_system_bus.svg
- https://www.realdigital.org/doc/a9fee931f7a172423e1ba73f66ca4081

01: Memory-Mapped IO

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University